# Итоговое задание по модулю 3

Дисциплина: ETL-процессы
Студент: Екатерина Ячменева
Преподаватель: Артём Озерков

## 1. Цель работы

Целью работы является реализация ETL-процесса с использованием Apache Airflow, MongoDB и PostgreSQL. В рамках работы выполняется генерация данных, репликация данных из MongoDB в PostgreSQL, трансформация данных и построение аналитических витрин.

## 2. Архитектура решения

MongoDB → Airflow ETL → PostgreSQL → аналитические витрины

MongoDB используется для хранения исходных JSON-данных. Apache Airflow используется как оркестратор ETL-процессов. PostgreSQL используется для хранения структурированных данных и аналитических витрин.

## 3. Развёртывание инфраструктуры

Проект развёрнут с использованием Docker Compose. В инфраструктуру входят следующие сервисы: PostgreSQL, MongoDB, Mongo Express, Apache Airflow и Jupyter Lab.

Доступ к сервисам:

Airflow UI — http://localhost:8080

Mongo Express — http://localhost:8082

Jupyter Lab — http://localhost:8888

## 4. Генерация данных в MongoDB

Для генерации тестовых данных используется Python-скрипт generate_mongo_data.py. Скрипт создаёт базу данных etl_db и заполняет коллекции user_sessions, event_logs и support_tickets.

Количество сгенерированных документов:
user_sessions — 1000
event_logs — 1000
support_tickets — 1000
Рисунок 1 — DAG генерации данных в Apache Airflow

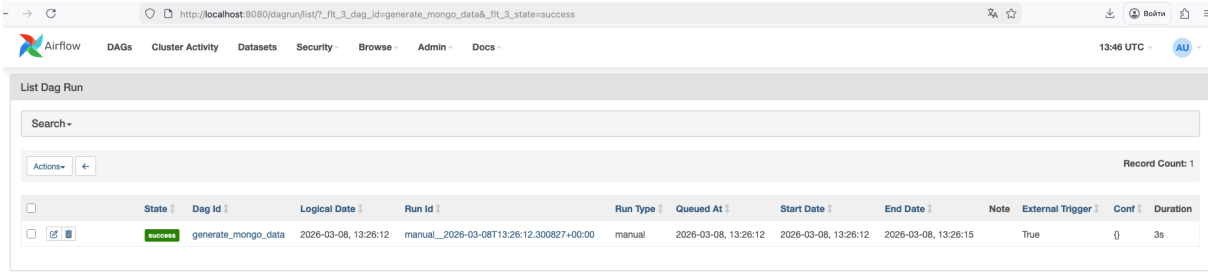



Рисунок 2 — успешное выполнение задачи generate_data_in_mongo

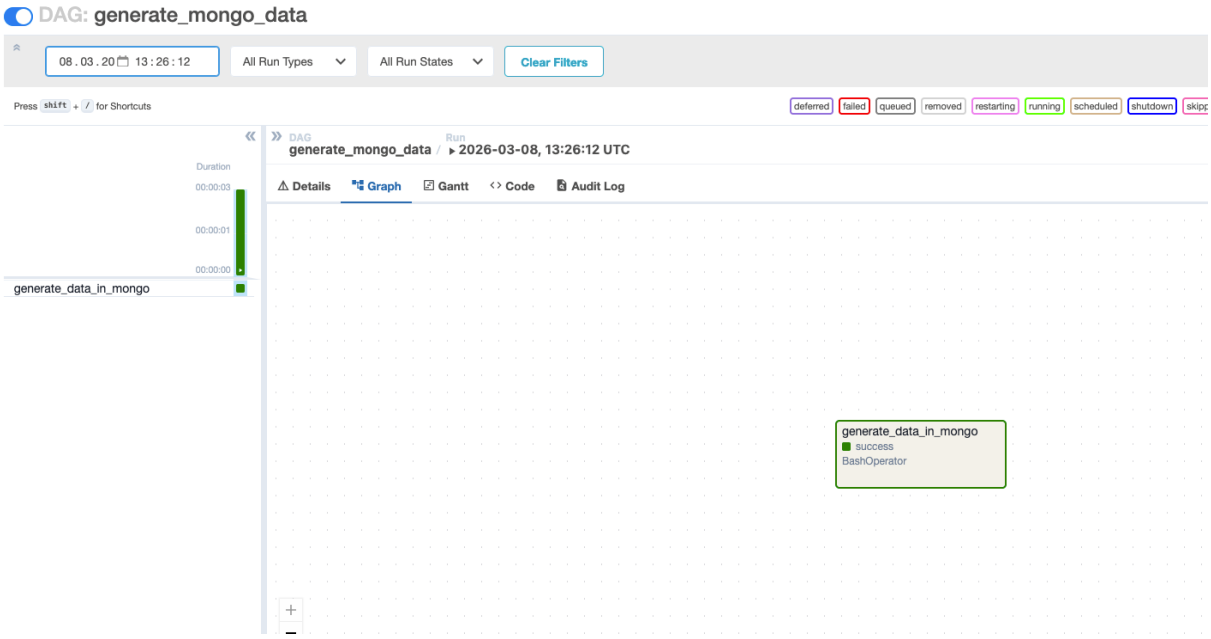


Рисунок 3 — коллекции MongoDB

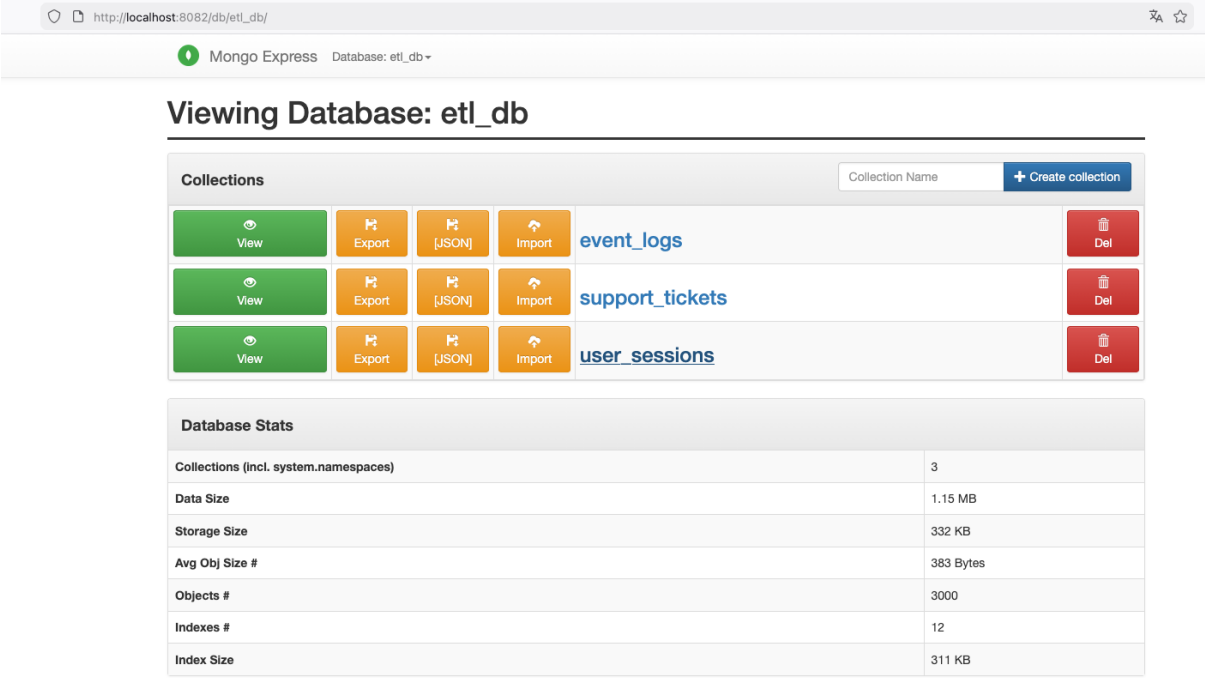
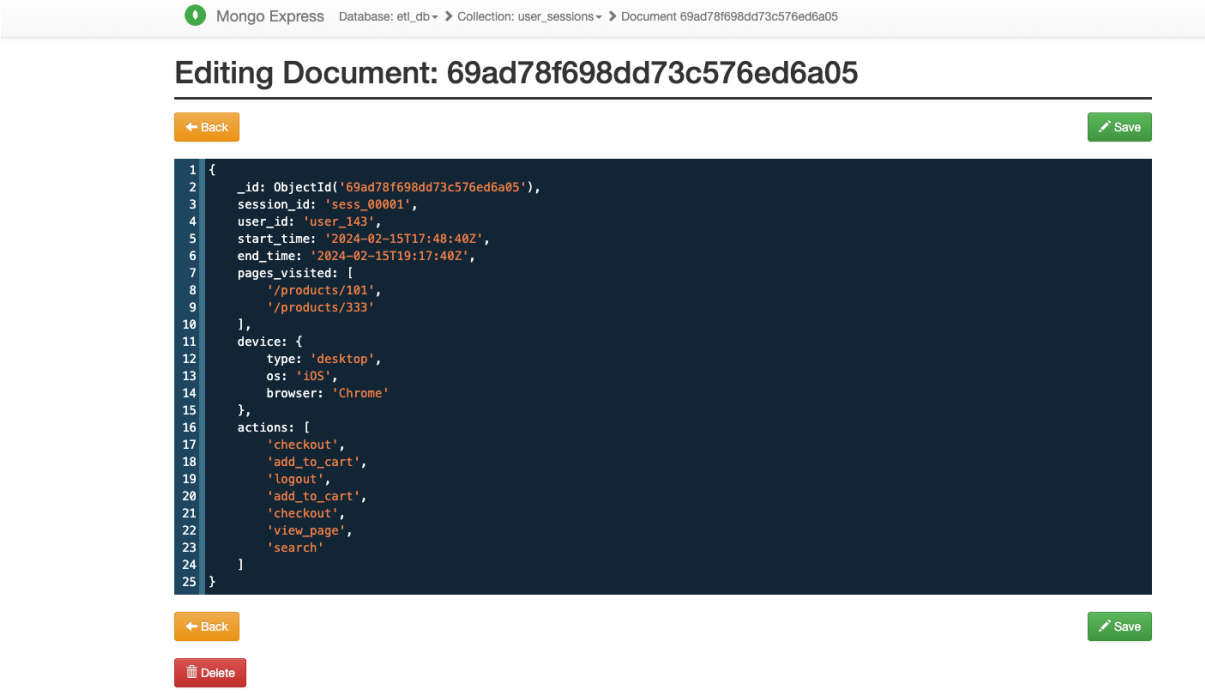



Рисунок 4 — пример JSON-документа в коллекции user_sessions (вставить скриншот).




## 5. Репликация данных из MongoDB в PostgreSQL

ETL-процесс состоит из трёх этапов: Extract (извлечение данных из MongoDB), Transform

(преобразование данных) и Load (загрузка данных в PostgreSQL).

## 6. Структура данных в PostgreSQL

В PostgreSQL создаются таблицы для хранения структурированных данных: user_sessions, session_pages, session_actions, event_logs, support_tickets и ticket_messages.

## 7 Пайплайн репликации данных

Для переноса данных из MongoDB в PostgreSQL был реализован ETL-пайплайн в Apache Airflow.

Пайплайн состоит из следующих этапов:

**Extract**

Извлечение данных из MongoDB из коллекций:

- user_sessions
- event_logs
- support_tickets

**Transform**

В процессе трансформации выполняются:

- преобразование ISO-дат в timestamp
- вычисление длительности сессии
- вычисление времени обработки тикета

- нормализация вложенных структур JSON
- разбиение массивов pages_visited и actions в отдельные таблицы

**Load**

После трансформации данные загружаются в PostgreSQL в таблицы схемы `etl`.

MongoDB

↓

Airflow DAG

↓

ETL Python

↓

PostgreSQL

Рисунок 5 DAG пайплайна репликации данных MongoDB

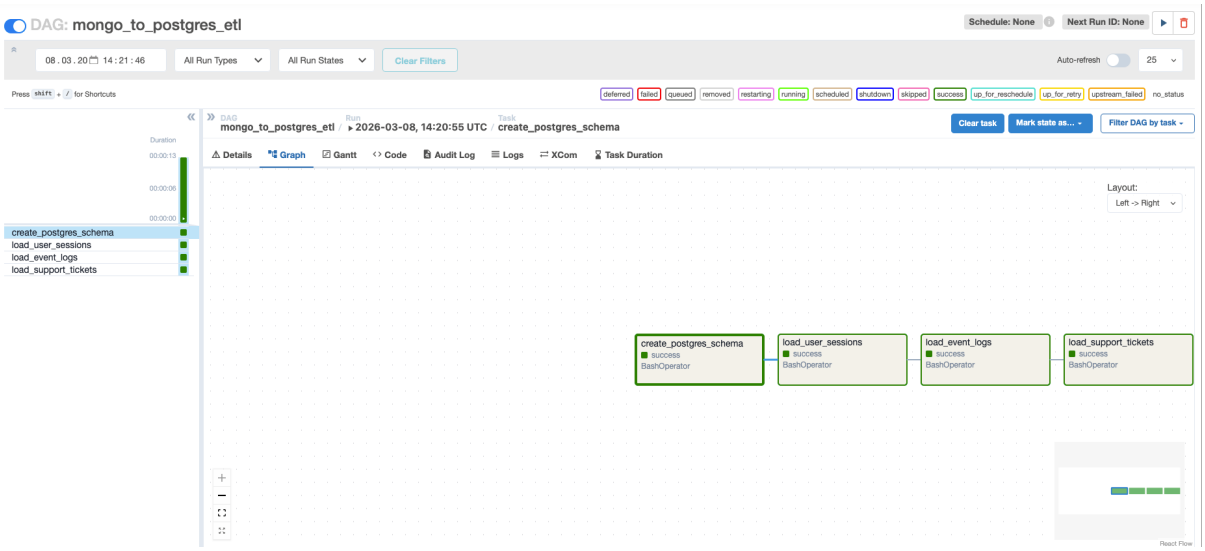



Рисунок 6 — Трансформация структуры данных MongoDB → PostgreSQL

MongoDB (исходные коллекции)

---

```
user_sessions
  ├─ session_id
  ├─ user_id
  ├─ start_time
  ├─ end_time
  ├─ pages_visited [array]
  ├─ actions [array]
  └─ device {type, os, browser}

event_logs
  ├─ event_id
  ├─ timestamp
```

```
  ├─ event_type
  └─ details {page, element}

support_tickets
  ├─ ticket_id
  ├─ user_id
  ├─ status
  ├─ issue_type
  ├─ messages [array]
  ├─ created_at
  └─ updated_at


        ↓  ETL (Airflow)
     Extract → Transform → Load
```

PostgreSQL (реляционная схема)

---

```
etl.user_sessions
  ├─ session_id (PK)
  ├─ user_id
  ├─ start_time
  ├─ end_time
  ├─ session_duration_sec
  ├─ device_type
  ├─ device_os
```

```
  └─ browser

etl.session_pages
  ├─ session_id (FK)
  ├─ page_order
  └─ page_url

etl.session_actions
  ├─ session_id (FK)
  ├─ action_order
  └─ action_name

etl.event_logs
  ├─ event_id (PK)
  ├─ event_time
  ├─ event_type
  ├─ page
  └─ element

etl.support_tickets
  ├─ ticket_id (PK)
  ├─ user_id
  ├─ status
  ├─ issue_type
  ├─ created_at
  ├─ updated_at
  └─ resolution_time_hours

etl.ticket_messages
```

├─ ticket_id (FK)
├─ message_order
├─ sender
├─ message_text
└─ message_time

Рисунок 7 проверка результата

```
(base) kataacmeneva@MacBook-Pro-kata etl_module3_final_task % docker exec -it de-postgres psql -U airflow -d airflow -c "\dt etl.*"
             List of relations
 Schema |      Name       | Type  |  Owner
--------+-----------------+-------+---------
 etl    | event_logs      | table | airflow
 etl    | session_actions | table | airflow
 etl    | session_pages   | table | airflow
 etl    | support_tickets | table | airflow
 etl    | ticket_messages | table | airflow
 etl    | user_sessions   | table | airflow
(6 rows)
```

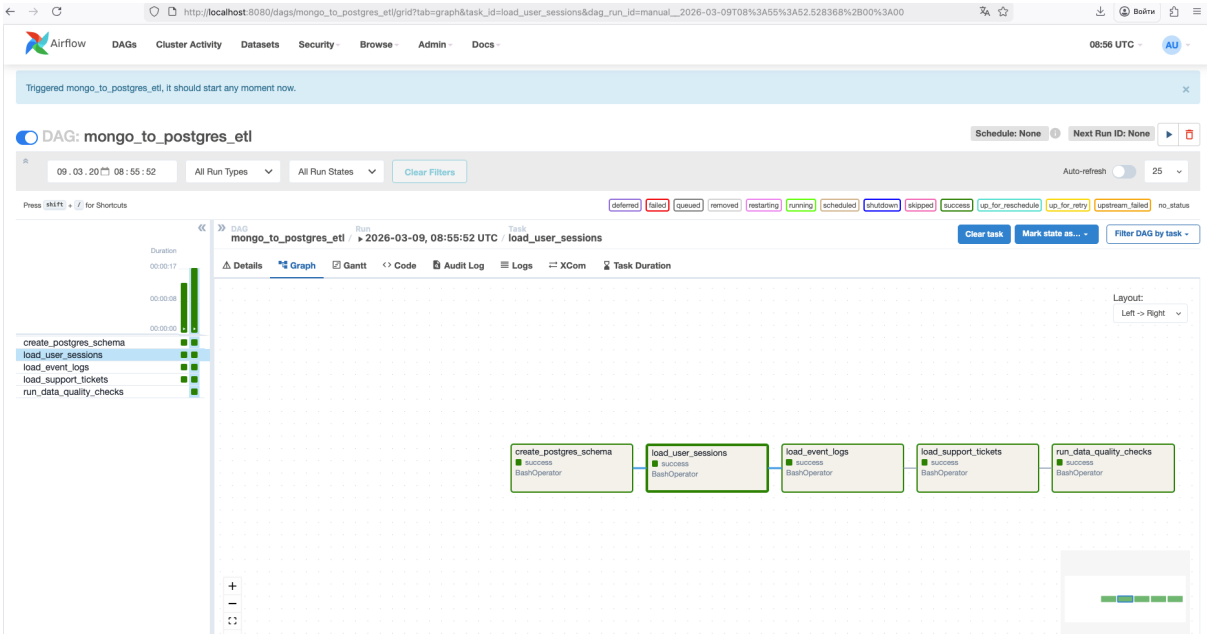
```
(base) kataacmeneva@MacBook-Pro-kata etl_module3_final_task % docker exec -it de-postgres psql -U airflow -d airflow -c "select count(*) from etl.user_sessions;"
docker exec -it de-postgres psql -U airflow -d airflow -c "select count(*) from etl.event_logs;"
docker exec -it de-postgres psql -U airflow -d airflow -c "select count(*) from etl.support_tickets;"
 count
-------
  1000
(1 row)

 count
-------
  1000
(1 row)

 count
-------
  1000
(1 row)
```

В ETL-пайплайн был добавлен этап контроля качества данных. После загрузки выполняются автоматические проверки на отсутствие дублей в основных таблицах и на наличие данных в загруженных таблицах.

Рисунок 8 проверка результата




## 7. Аналитические витрины

Создаются две аналитические витрины: витрина активности пользователей и витрина эффективности работы поддержки.

Рисунок 9 — DAG построения аналитических витрин.

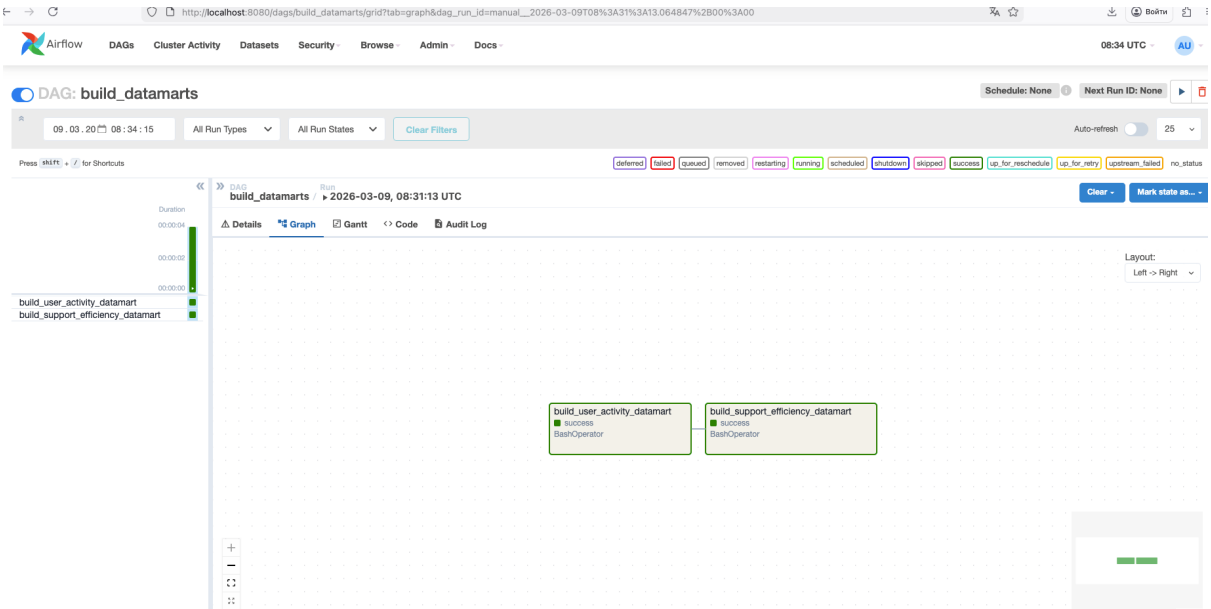


Рисунок 10 — аналитические витрины PostgreSQL.

```
airflow=# \dt etl.dm_*
                List of relations
 Schema |         Name          | Type  |  Owner
--------+-----------------------+-------+---------
 etl    | dm_support_efficiency | table | airflow
 etl    | dm_user_activity      | table | airflow
(2 rows)

airflow=# select * from etl.dm_user_activity limit 10;
 user_id  | sessions_count | avg_session_duration_sec | pages_visited_count | actions_count
----------+----------------+--------------------------+---------------------+---------------
 user_1   |              4 |                  3525.00 |                  18 |            21
 user_10  |              8 |                  3555.00 |                  35 |            51
 user_100 |              5 |                  3156.00 |                  28 |            31
 user_101 |              8 |                  5280.00 |                  39 |            34
 user_102 |              3 |                  3180.00 |                  10 |            13
 user_103 |              7 |                  4362.86 |                  28 |            33
 user_104 |              7 |                  3934.29 |                  30 |            38
 user_105 |              6 |                  5230.00 |                  33 |            37
 user_106 |              8 |                  3892.50 |                  43 |            36
 user_107 |              4 |                  5430.00 |                  20 |            21
(10 rows)
```

```
airflow=# select * from etl.dm_support_efficiency limit 10;
   issue_type     |   status    | tickets_count | avg_resolution_time_hours
------------------+-------------+---------------+---------------------------
 account          | closed      |            62 |                     62.18
 account          | in_progress |            65 |                     41.17
 account          | open        |            62 |                     12.34
 delivery         | closed      |            66 |                     64.06
 delivery         | in_progress |            73 |                     37.11
 delivery         | open        |            68 |                     13.38
 payment          | closed      |            58 |                     68.95
 payment          | in_progress |            76 |                     36.61
 payment          | open        |            63 |                     12.27
 product_quality  | closed      |            71 |                     66.56
(10 rows)

airflow=#
```

В рамках расширения функциональности была реализована дополнительная аналитическая витрина `etl.dm_popular_pages`, позволяющая определять наиболее посещаемые страницы сервиса по данным пользовательских сессий. Витрина `dm_user_activity` отражает агрегированную активность пользователей: количество сессий, среднюю длительность сессии,

общее число посещённых страниц и совершённых действий.

Для более детального анализа популярности контента дополнительно была построена витрина `dm_popular_pages`, позволяющая определить наиболее посещаемые страницы сервиса.

Рисунок 11

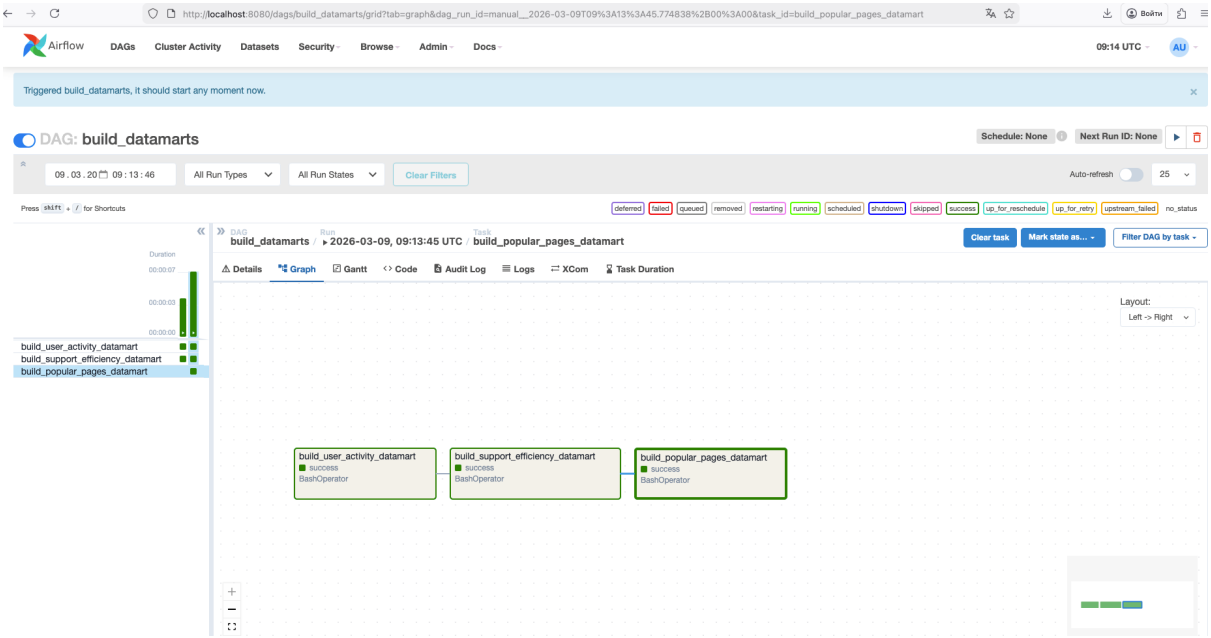



Рисунок 12 Проверка результата

```
(base) kataacmeneva@MacBook-Pro-kata etl_module3_final_task % docker exec -it de-postgres psql -U airflow -d airflow -c
                List of relations
 Schema |          Name           | Type  |  Owner
--------+-------------------------+-------+---------
 etl    | dm_popular_pages        | table | airflow
 etl    | dm_support_efficiency   | table | airflow
 etl    | dm_user_activity        | table | airflow
(3 rows)
```

```
(base) kataacmeneva@MacBook-Pro-kata etl_module3_final_task % docker exec -it de-postgres psql -U airflow -d airflow -c
   page_url     | visits_count
---------------+--------------
 /products/101 |          529
 /products/205 |          516
 /support      |          515
 /cart         |          500
 /home         |          497
 /checkout     |          496
 /profile      |          487
 /products     |          483
 /catalog      |          478
 /products/333 |          474
(10 rows)
```

## 8. DAG `partition_event_logs` — создание партиционированной таблицы событий

Airflow используется для автоматизации генерации данных, репликации данных и построения аналитических витрин.
Для повышения эффективности хранения и обработки событийных данных был реализован отдельный DAG `partition_event_logs`. Его задача — создать в PostgreSQL партиционированную таблицу `etl.event_logs_part`, предназначенную для хранения событий пользователей с разделением данных по временным диапазонам.

Партиционирование выполняется по полю времени события (`event_time`) с использованием диапазонного (`RANGE`) метода. Для таблицы создаются отдельные партиции по месяцам (например, `event_logs_2024_01`, `event_logs_2024_02`, `event_logs_2024_03`).
После создания структуры в партиционированную таблицу копируются данные из основной таблицы `etl.event_logs`.

DAG состоит из одной задачи `create_partitioned_event_logs`, которая запускает Python-скрипт

`create_partitioned_event_logs.py`. Скрипт выполняет SQL-команды для:

- создания партиционированной таблицы `etl.event_logs_part`,
- создания месячных партиций,
- переноса существующих данных из таблицы `etl.event_logs` в новую структуру.

Использование партиционирования позволяет повысить производительность аналитических запросов по временным диапазонам и облегчает дальнейшее масштабирование системы хранения событий.

Рисунок 13

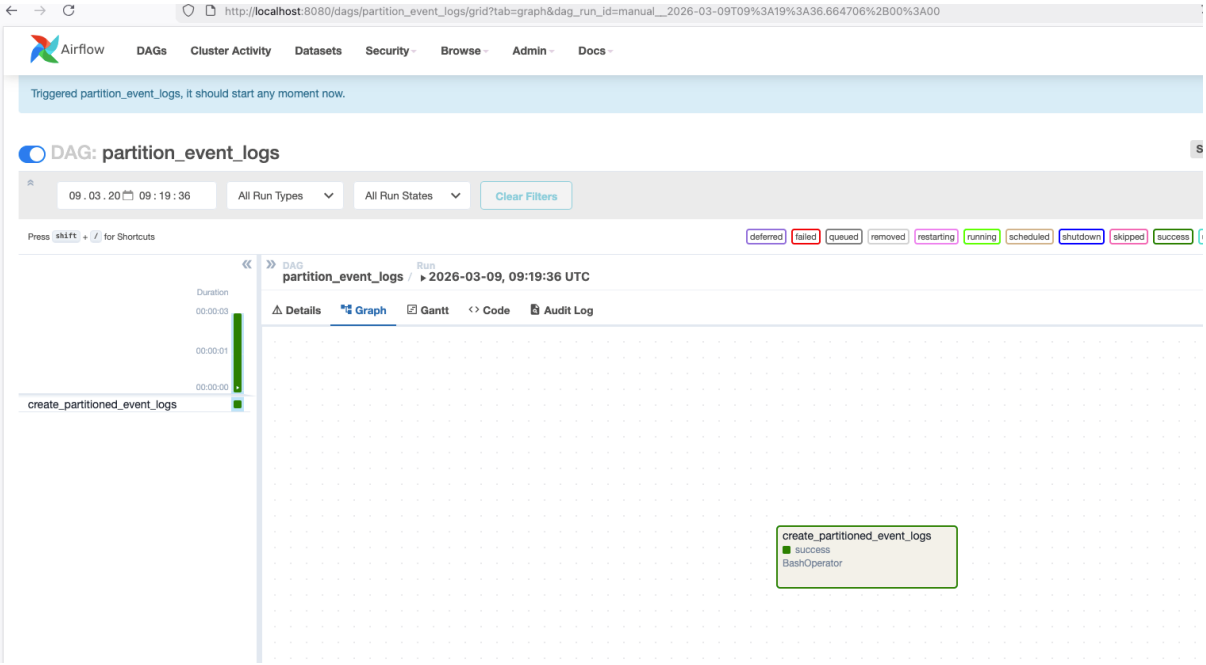


Рисунок 14 Реализовано партиционирование таблицы событий `event_logs` по диапазону дат.

```
(base) kataacmeneva@MacBook-Pro-kata etl_module3_final_task % docker exec -it de-postgres psql -U airflow -d airflow
psql (14.20 (Debian 14.20-1.pgdg13+1))
Type "help" for help.

airflow=# \dt etl.event_logs*
                    List of relations
 Schema |        Name        |       Type        |  Owner
--------+--------------------+-------------------+---------
 etl    | event_logs         | table             | airflow
 etl    | event_logs_2024_01 | table             | airflow
 etl    | event_logs_2024_02 | table             | airflow
 etl    | event_logs_2024_03 | table             | airflow
 etl    | event_logs_part    | partitioned table | airflow
(5 rows)
```

## 9. Заключение

В ходе выполнения работы реализован полноценный ETL-процесс, позволяющий генерировать данные, переносить их между базами данных и строить аналитические витрины.